Amortized Analysis

- The problem domains vary widely, so this approach is not tied to any single data structure
- The goal is to guarantee the average performance of each operation *in the worst case*
- Three methods
  - aggregate method
  - accounting method
  - potential method
- Two problems
  - stack operations, including a multipop
  - binary counter using the increment operation

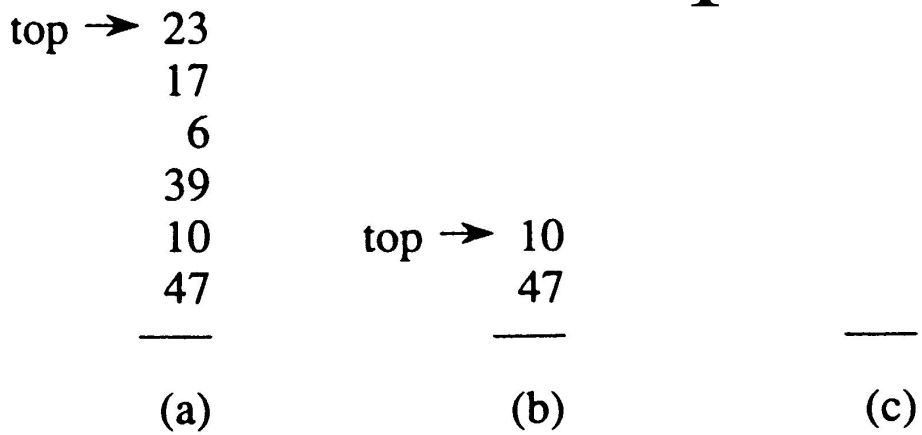
Stack Operations

- Goal
  - find worst case time, T(n), for n operations
  - the amortized cost is T(n) / n
- Push and Pop
  - Push(x, S) has complexity O(1)
  - Pop(S) returns popped object, complexity is O(1)
- Multipop operation

```
MULTIPOP(S, k)
while not STACK-EMPTY(S) and k ≠ 0
    do POP(S)
       k ← k − 1
```

  - complexity is min(s,k) where s is the stack size

A Simple Analysis



- Start with empty stack
  - Stack can be no larger than O(n)
  - So a multipop operation could have complexity O(n)
  - Since there are n operations, an upper bound on complexity is $O(n^2)$
- Although this is a valid upper bound, it grossly overestimates the upper bound

An Amortized Analysis

- Claim - *any* sequence of push, pop, multipop has at most worst case complexity of O(n)
  - each object can be popped at most one time for each time it is pushed
  - the number of push operations is O(n) at most
  - so the number of pops, either from pop or multipop, is at most O(n)
  - the overall complexity is O(n)
- The amortized cost is O(n)/n = O(1)
- Question - would this still be true if we had a multipush and a multipop?

Incrementing Binary Numbers

- Assume bits A[k-1] to A[0]

$$x = \sum_{i=0}^{k-1} A[i] * 2^i$$

- Incrementing starting from 0

```
INCREMENT(A)
i ← 0
while i < length[A] and A[i] = 1
      do A[i] ← 0
         i ← i + 1
if i < length[A]
    then A[i] ← 1
```

Counter value	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

- The diagram to the right counts the number of bit flips; for the first 16 numbers the total cost is 31; in general, for n increments, what will the complexity be?

A Simple Analysis

- We measure cost as the number of bit flips
  - some operations only flip one bit
  - other operations ripple through the number and flip many bits
  - what is the average cost per operation?
- A cursory analysis
  - worst case for the increment operation is O(k)
  - a sequence of n increment operations would have worst case behavior of O(n k)
- Although this is an upper bound, it is not very tight

An Amortized Analysis

- Not all bits are flipped each iteration
  - A[0] is flipped every iteration; A[1] every other iteration
  - A[2] every fourth, and so forth
  - for i > ⌊lg n⌋, the bit A[i] never flips
  - summing all the bit flips we have

$$\sum_{i=0}^{\lfloor \lg n \rfloor} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n$$

- So it turns out the worst case is bounded by O(n)
- Therefore O(n) / n is only O(1) !

The Accounting Method

- Different charges are assigned to different operations
  - overcharges result in a credit
  - credits can be used later to help perform other operations whose amortized cost is less than actual
  - this is very different than the aggregate method in which all operations have the same amortized cost
- Choice of amortized amounts
  - the total amortized cost for any sequence must be an upper bound on the actual costs
  - the total credit assignment must be nonnegative
  - so the amortized amounts selected must never result in a negative credit

Stack Operations

Operation	Actual cost	Amortized cost
Push	1	2
Pop	1	0
Multipop	min(s,k)	0

- Rationale
  - since we start with an empty stack, pushes must be done first and this builds up the amortized credit
  - all pops are charged against this credit; there can never be more pops (of either type) than pushes
  - therefore the total amortized cost is O(n)

Incrementing a Binary Counter

- Amortized cost
  - 2 for setting a bit to 1
  - 0 for setting a bit to 0
  - the credits for any number are the number of 1 bits

```
INCREMENT(A)
1  i ← 0
2  while i < length[A] and A[i] = 1
3        do A[i] ← 0
4           i ← i + 1
5  if i < length[A]
6     then A[i] ← 1
```

- Analysis of the increment operation
  - the while loop resetting bits is charged against credits
  - only one bit is set in line 6, so the total charge is 2
  - since the number of 1's is never negative, the amount of credit is also never negative
  - the total amortized cost for n increments is O(n)

The Potential Method

- Potential is the accumulation of credits
  - it is associated with the entire data structure rather than individual objects; $\varphi(D_i)$ is a real number associated with the structure D_i
  - the amortized cost is $$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) \ .$$
  - the total amortized cost is $$\begin{aligned} \sum_{i=1}^{n} \hat{c}_i &= \sum_{i=1}^{n} (c_i + \Phi(D_i) - \Phi(D_{i-1})) \\ &= \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0) \ . \end{aligned}$$
  - if we insure that $\varphi(D_i) >= \varphi(D_0)$ then the potential never becomes negative
  - it is often convenient to let $\varphi(D_0) = 0$ then it only needs to be shown that all $\varphi(D_i)$ are nonnegative

Stack Operations - 1

- The potential function is the size of the stack

  the total amortized cost of n operations with respect to φ is an upper bound for the actual cost

$$\Phi(D_i) \geq 0$$
$$= \Phi(D_0)$$

- The push operation

$$\Phi(D_i) - \Phi(D_{i-1}) = (s+1) - s$$
$$= 1$$

so $\hat{c}_i' = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 2$

- The pop operation
  - the difference in potential for the pop operation is -1
  - so the amortized cost is 1 + (-1) = 0

Stack Operations - 2

- The multipop operation

$$\Phi(D_i) - \Phi(D_{i-1}) = -k'$$ where k' = min(k,s)

so
$$\begin{aligned} \hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= k' - k' \\ &= 0 \end{aligned}$$

- Total amortized cost
  - Each operation has an amortized cost of O(1)
  - For n operations, the total amortized cost is O(n)
  - Since the potential function meets all of our requirements, the total cost is a valid upper bound

Incrementing a Binary Counter - 1

- Potential function - the number of 1s in the count after the ith operation
- Amortized cost of the ith increment operation
  - suppose that t_i bits are reset and one bit is set

$$\begin{aligned}\Phi(D_i) - \Phi(D_{i-1}) &\leq (b_{i-1} - t_i + 1) - b_{i-1} \\ &= 1 - t_i\end{aligned}$$

  - since the actual cost is $t_i + 1$, we have

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &\leq (t_i + 1) + (1 - t_i) \\ &= 2\end{aligned}$$

- Therefore, for n operations, the total cost is O(n)

Incrementing a Binary Counter - 2

- Suppose the counter is not initially zero
  - there are b_0 initial 1s and after n increments b_n 1s

$$\sum_{i=1}^{n} c_i = \sum_{i=1}^{n} \hat{c}_i - \Phi(D_n) + \Phi(D_0) \ .$$

  - But the amortized cost for c are each 2, so

$$\begin{aligned} \sum_{i=1}^{n} c_i &\leq \sum_{i=1}^{n} 2 - b_n + b_0 \\ &= 2n - b_n + b_0 \end{aligned}$$

- Total cost

  since $b_0 < k$, if we executed at least $n = \Omega(k)$ increment operations, then the total cost is no more than $O(n)$ no matter what the starting value of the counter

Dynamic Tables

- Dynamic tables expand and contract in size according to the amount of data being held
  - the organization of the table is unimportant, for simplicity we will show an array structure
  - the load factor $\alpha(T)$ of a nonempty table equals the number of items in the table divided by the table size
  - an empty table has size 0 and load factor 1
  - when we try to insert an item into a full table, we will double the size of the table
  - when the load factor is too low we will contract the table
- Java vectors act in this way (at least for expansion)

The Insert Operation

Initially num(T)=size(T)=0

Lines 1-3 handle insertion into the empty table

Lines 10-11 occur for every insert operation

Lines 4-9 allow for table expansion

```
TABLE-INSERT(T, x)
 1  if size[T] = 0
 2      then allocate table[T] with 1 slot
 3           size[T] ← 1
 4  if num[T] = size[T]
 5      then allocate new-table with 2 · size[T] slots
 6           insert all items in table[T] into new-table
 7           free table[T]
 8           table[T] ← new-table
 9           size[T] ← 2 · size[T]
10  insert x into table[T]
11  num[T] ← num[T] + 1
```

- A simple analysis
  - the expansion is most expensive, for the ith insertion with expansion we could have $c_i = i$
  - for n insertions, worst case is O(n) for each and $O(n^2)$ total; but this bound can be improved

Using the Aggregate Method

- Table expansion only occurs at powers of 2

$$c_i = \begin{cases} i & \text{if } i-1 \text{ is an exact power of 2 ,} \\ 1 & \text{otherwise .} \end{cases}$$

The total cost of n TABLE-INSERT operations is therefore

$$\begin{aligned} \sum_{i=1}^{n} c_i &\le n + \sum_{j=0}^{\lfloor \lg n \rfloor} 2^j \\ &< n + 2n \\ &= 3n \end{aligned}$$

- The amortized cost for a single operation is 3

Using the Accounting Method

- We set the amortized cost for insert at 3
  - intuitive, one for inserting itself into current table
  - one for moving itself when the table is expanded
  - one for moving another item than has already been moved once the table is expanded
- After an expansion to size m for the table
  - there is no available credit
  - filling the available slots requires 1 actual cost
  - a second credit is associated with the item for a copy in the future; a third credit goes to copy an existing item
- Therefore, the total amortized cost is 3n

Using the Potential Method

- The potential function is $\Phi(T) = 2 \cdot num[T] - size[T]$
  - immediately after expansion, num[T] = size[T]/2 so the potential is 0 (all credits used for expanion)
  - immediately before expansion, num[T] = size[T] so the potential is num[T] which will handle the copying
  - since num[T] >= size[T]/2, the potential in nonnegative

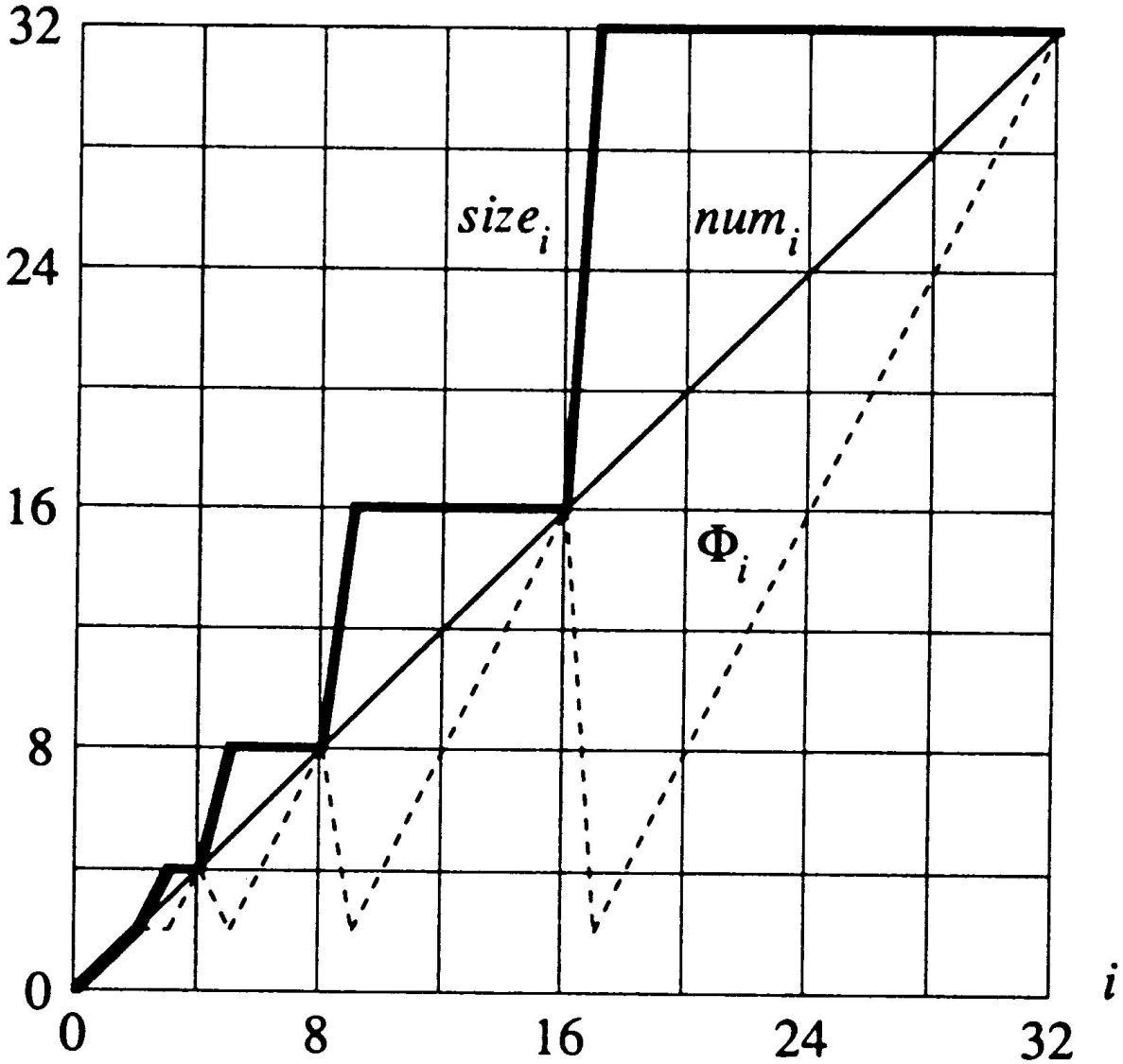
If the ith TABLE-INSERT operation does not trigger an expansion, then $size_i = size_{i-1}$ and the amortized cost of the operation is

$$\begin{aligned}
\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= 1 + (2 \cdot num_i - size_i) - (2 \cdot num_{i-1} - size_{i-1}) \\
&= 1 + (2 \cdot num_i - size_i) - (2(num_i - 1) - size_i) \\
&= 3 .
\end{aligned}$$

If the ith operation does trigger an expansion, then $size_i/2 = size_{i-1} = num_i - 1$, and the amortized cost of the operation is

$$\begin{aligned}
\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= num_i + (2 \cdot num_i - size_i) - (2 \cdot num_{i-1} - size_{i-1}) \\
&= num_i + (2 \cdot num_i - (2 \cdot num_i - 2)) - (2(num_i - 1) - (num_i - 1)) \\
&= num_i + 2 - (num_i - 1) \\
&= 3
\end{aligned}$$

Relationships between Values



Allowing for Table Contractions

- Contracting when half full - not a good strategy
  - suppose we perform I, D, D, I, I, D, D, I, I, etc. after n/2 insertions
  - these operations would alternately expand and contract the table leading to a complexity of $O(n^2)$
  - we do not have enough credits to pay for expansion and contraction alternating so quickly
- We will expand at 1/2 full but only contract at 1/4
  - we earn credits going from 1/2 to 1/4 to make contraction possible
  - after the contraction is completed, the load factor is now 1/2 which would allow another contraction

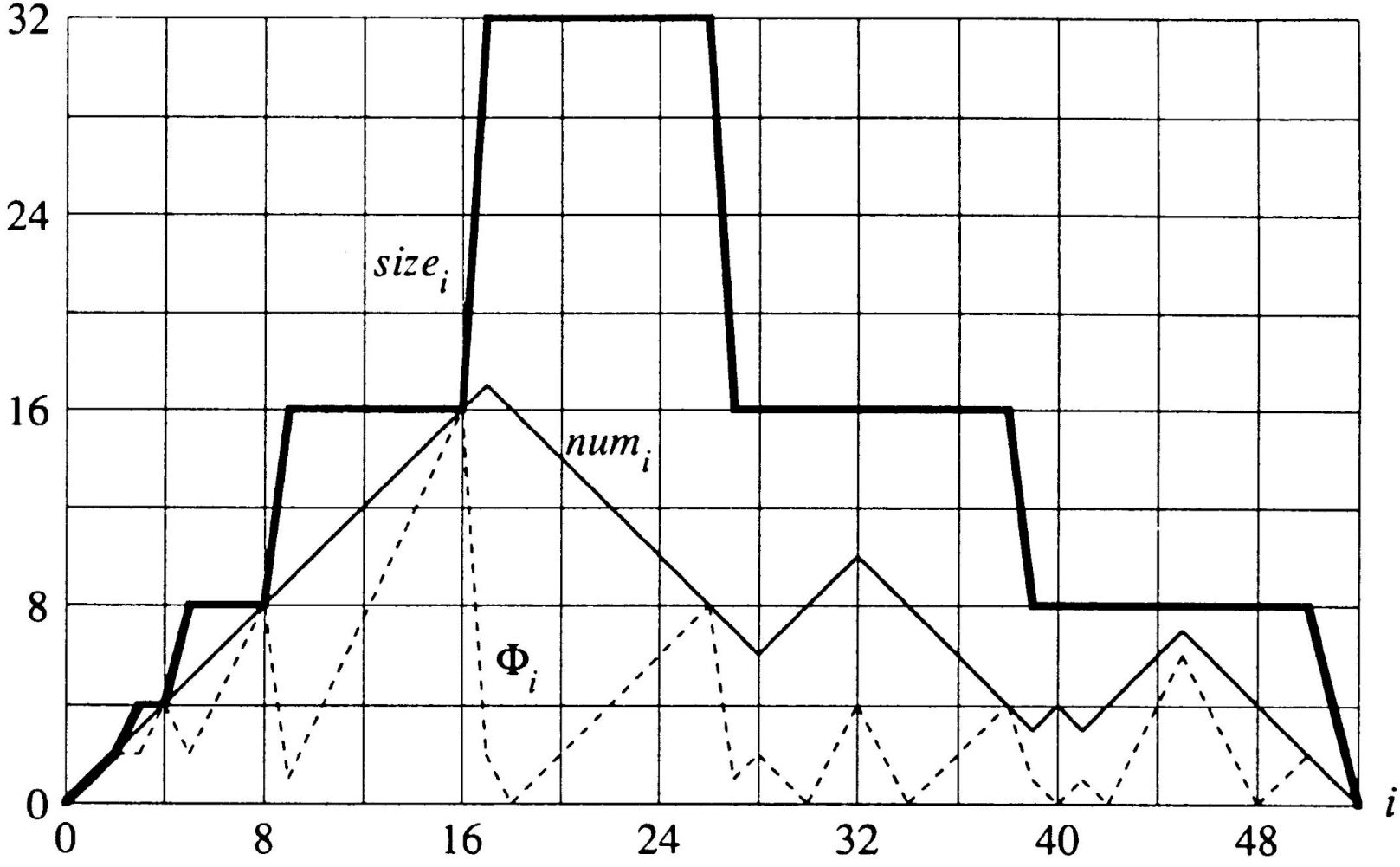
Using the Potential Method

- The potential function is

$$\Phi(T) = \begin{cases} 2 \cdot num[T] - size[T] & \text{if } \alpha(T) \geq 1/2 \ , \\ size[T]/2 - num[T] & \text{if } \alpha(T) < 1/2 \end{cases}$$

  - The potential is 0 after expansion or contraction
  - It builds while the load factor increases towards 1 or decreases towards 1/4
  - the potential is never negative, so the total amortized cost can be an upper bound
  - when the load factor is 1, the potential is num[T], thus the potential can pay for an expansion
  - when the load factor is 1/4, the potential is also num[T], so it can pay for a contraction

Sample Behavior



Amortized Cost for Insertion

- If α_{i-1} >= 1/2, the analysis is like before with an amortized cost of 3 at most
- If α_i < 1/2 and α_{i-1} < 1/2 then

$$
\begin{aligned}
\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= 1 + (size_i/2 - num_i) - (size_{i-1}/2 - num_{i-1}) \\
&= 1 + (size_i/2 - num_i) - (size_i/2 - (num_i - 1)) \\
&= 0
\end{aligned}
$$

If $\alpha_{i-1} < 1/2$ but $\alpha_i \geq 1/2$, then

$$
\begin{aligned}
\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= 1 + (2 \cdot num_i - size_i) - (size_{i-1}/2 - num_{i-1}) \\
&= 1 + (2(num_{i-1} + 1) - size_{i-1}) - (size_{i-1}/2 - num_{i-1}) \\
&= 3 \cdot num_{i-1} - \frac{3}{2} size_{i-1} + 3 \\
&= 3\alpha_{i-1} size_{i-1} - \frac{3}{2} size_{i-1} + 3 \\
&< \frac{3}{2} size_{i-1} - \frac{3}{2} size_{i-1} + 3 \\
&= 3
\end{aligned}
$$

- Thus the amortized cost of insert is at most 3

Amortized Cost of Deletion

- If $\alpha_{i-1} < 1/2$ but there is no contraction then

$$\begin{aligned}
\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= 1 + (size_i/2 - num_i) - (size_{i-1}/2 - num_{i-1}) \\
&= 1 + (size_i/2 - num_i) - (size_i/2 - (num_i + 1)) \\
&= 2
\end{aligned}$$

- If $\alpha_{i-1} < 1/2$ and the ith operation causes a contraction then

$$\begin{aligned}
\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= (num_i + 1) + (size_i/2 - num_i) - (size_{i-1}/2 - num_{i-1}) \\
&= (num_i + 1) + ((num_i + 1) - num_i) - ((2 \cdot num_i + 2) - (num_i + 1)) \\
&= 1
\end{aligned}$$

- If $\alpha_{i-1} > 1/2$ the operation is bounded by a constant (this is exercise 18.4-3)